

Competitive Programming

Section 22: One-Dimensional DP

Competitive Programming Course

2026-08-03

Table of contents

1	Why dynamic programming starts here	3
2	The DP checklist	4
3	Recognition	5
4	Example 1: counting stairs	6
5	Example 2: take or skip days	7
6	Example 3: minimum energy route	8
7	Vocabulary	9
7.1	State	9
7.2	Base case	9
7.3	Transition	9
7.4	Computation order	9
8	Counting, minimizing, maximizing	11
9	Modulo in counting DP	12
10	Rolling arrays	13
11	Local exercises	14
11.1	A. Broken Stairs	14
11.2	B. Training Score	14
11.3	C. Energy Route	14
12	C++ implementation notes	16
13	Python implementation notes	17
14	Proof habits for DP	18
15	Common mistakes	19
16	Local judge workflow	20
17	Submission checklist	21
18	Exercises	22

1 Why dynamic programming starts here

Many contest problems look like this:

```
There is a sequence.  
You make local choices.  
The same suffix/prefix question appears again and again.  
Brute force branches exponentially.
```

Dynamic programming, or DP, is the habit of naming those repeated subproblems, solving them once, and reusing their answers.

One-dimensional DP is the cleanest starting point because the state is usually one index:

```
dp[i] = answer for the prefix ending at i
```

or:

```
dp[i] = answer when we stand at position i
```

2 The DP checklist

Before writing code, fill this in:

```
State:
    What does dp[i] mean?

Base case:
    Which dp values are known immediately?

Transition:
    How do earlier states produce dp[i]?

Order:
    In what order must states be computed?

Answer:
    Which state is printed?
```

If the state sentence is vague, the implementation will usually be wrong.

3 Recognition

One-dimensional DP is likely when:

Statement signal	DP shape
walk along a line with allowed jumps	<code>dp[position]</code>
count ways to reach a target	sum of previous states
minimize energy/cost along ordered positions	min over previous states
maximize reward with local restrictions	max of take/skip states
choice at day <code>i</code> depends on recent days	prefix DP

The key word is not “minimum” or “maximum”. Greedy and graph problems also use those words. The stronger signal is:

The problem asks for the best/count over many possible histories, but the future only needs a small summary of the past.

4 Example 1: counting stairs

You can climb 1 or 2 steps. Count ways to reach step n .

State:

```
dp[i] = number of ways to land on step i
```

Base:

```
dp[0] = 1
```

There is one way to be at the start before climbing: do nothing.

Transition:

```
dp[i] = dp[i - 1] + dp[i - 2]
```

because the last jump into i came either from $i-1$ or from $i-2$.

Trace:

```
i:      0  1  2  3  4  5
dp:      1  1  2  3  5  8
```

With broken step 3, set $dp[3] = 0$:

```
i:      0  1  2  3  4  5
broken:  .  .  .  X  .  .
dp:      1  1  2  0  2  2
```

5 Example 2: take or skip days

You have daily scores. You may not take two adjacent days.

State:

```
dp[i] = best score using only the first i days
```

For day i , there are two cases:

```
skip day i:
    dp[i - 1]

take day i:
    dp[i - 2] + score[i]
```

So:

```
dp[i] = max(dp[i - 1], dp[i - 2] + score[i])
```

This recurrence is valuable because it avoids remembering the whole chosen set. The prefix answer $dp[i]$ is enough.

Trace for scores:

day:	1	2	3	4	5	6
score:	5	1	2	10	6	2
dp:	5	5	7	15	15	17

Answer: 17.

6 Example 3: minimum energy route

There are platforms in a line. You can jump from j to i if $i - j \leq k$. The jump costs $\text{abs}(h[j] - h[i])$.

State:

```
dp[i] = minimum energy needed to reach platform i
```

Base:

```
dp[0] = 0
```

Transition:

```
dp[i] = min over j in [i-k, i-1] of dp[j] + abs(h[j] - h[i])
```

ASCII view:

```
          can jump to i
        j  j+1  ...  i-1  i
        |   |       |   |
dp[j] + cost(j,i)  candidate answer
```

Compute i from left to right because every candidate j is smaller than i , so $dp[j]$ is already known.

7 Vocabulary

7.1 State

A state is the information that identifies a subproblem.

Good:

```
dp[i] = minimum energy to reach platform i
```

Bad:

```
dp[i] = answer at i
```

The bad version does not say what “answer” means.

7.2 Base case

A base case is a state known without using the recurrence. It anchors the DP.

Common one-dimensional base cases:

```
dp[0] = 0      for minimum cost from the start
dp[0] = 1      for counting ways to have built nothing
dp[1] = value  for first-item problems
```

7.3 Transition

A transition explains how smaller subproblems combine into a larger one.

Typical forms:

```
counting:    dp[i] = sum(...)
minimizing:  dp[i] = min(...)
maximizing:  dp[i] = max(...)
```

7.4 Computation order

The order must compute dependencies first.

For this section:

```
for i from left to right:  
    compute dp[i] from smaller indices
```

Later DP sections will use grids, intervals, trees, and bitmasks, where the order is less obvious.

8 Counting, minimizing, maximizing

The operation in the recurrence follows the output:

Goal	Transition operation
count all valid ways	sum
minimum cost	min
maximum score	max
existence	boolean or

But the operation alone is not enough. You still need the right state.

For example:

```
minimum energy to reach i
```

is different from:

```
minimum energy among all paths that have passed through i
```

The first one is useful because the future only needs the cost to stand at *i*. The second is vague and harder to transition.

9 Modulo in counting DP

Counting answers can be enormous. Problems often ask for the answer modulo 1_000_000_007.

Rule:

```
apply modulo after every addition or multiplication that may grow large
```

For Broken Stairs:

```
dp[i] = (dp[i - 1] + dp[i - 2]) % MOD
```

Modulo does not change which paths are valid. It only keeps the stored number small.

10 Rolling arrays

If `dp[i]` only depends on the previous few states, you do not always need the whole array.

For Training Score:

```
dp[i] depends only on dp[i-1] and dp[i-2]
```

So we can store:

```
prev2 = dp[i-2]
prev1 = dp[i-1]
cur    = dp[i]
```

Update:

```
cur = max(prev1, prev2 + score)
prev2 = prev1
prev1 = cur
```

Use rolling arrays only after the recurrence is clear. Compressing too early is a common source of off-by-one bugs.

11 Local exercises

11.1 A. Broken Stairs

State:

```
dp[i] = number of valid ways to reach step i
```

Transition:

```
if i is broken:
    dp[i] = 0
else:
    dp[i] = dp[i-1] + dp[i-2]
```

This trains counting DP and modulo.

11.2 B. Training Score

State:

```
dp[i] = maximum score using the first i days
```

Transition:

```
dp[i] = max(skip day i, take day i)
        = max(dp[i-1], dp[i-2] + a[i])
```

This trains maximizing DP and rolling-array compression.

11.3 C. Energy Route

State:

```
dp[i] = minimum energy needed to reach platform i
```

Transition:

```
dp[i] = min(dp[j] + abs(h[j] - h[i])) for i-k <= j < i
```

This trains minimization over several previous states.

12 C++ implementation notes

Use `long long` for costs and scores:

```
vector<long long> dp(n, INF);
```

For counting modulo:

```
const long long MOD = 1000000007LL;
```

For impossible or not-yet-computed minimum states, choose an infinity much larger than any possible answer:

```
const long long INF = (1LL << 62);
```

13 Python implementation notes

Python integers do not overflow, but you still need modulo for required output.

For large inputs:

```
data = list(map(int, sys.stdin.buffer.read().split()))
```

For minimum DP:

```
inf = 10**30  
dp = [inf] * n
```

14 Proof habits for DP

Most DP correctness proofs use induction.

Template:

Claim: `dp[i]` has the intended meaning after it is computed.

Base: prove the first state(s).

Inductive step:

- assume all earlier states are correct.
- take an arbitrary valid solution for state `i`.
- inspect its last decision and show that it matches one case in the transition; therefore no valid solution is omitted.
- show that every case in the transition produces a valid solution.

Here, the “last decision” is the part removed to obtain a smaller DP state. In one-dimensional DP, it is often:

- last jump
- last selected day
- last previous position

15 Common mistakes

Leaving the state meaning implicit.

Using `dp[0] = 0` in a counting problem where empty construction should count as 1.

Forgetting to handle `i-2` when `i = 1`.

Taking modulo in max/min DP where it does not belong.

Using `int` in C++ for large scores or costs.

Compressing memory before testing the array version.

16 Local judge workflow

Run this section:

```
CP_TARGET=solution python3 sections/22_one_dimensional_dp/check.py
```

Run everything:

```
CP_TARGET=solution python3 tools/check_all.py
```

17 Submission checklist

Before submitting a DP solution:

```
Can I say exactly what dp[i] means?  
Are base cases initialized before transitions?  
Does my loop order compute dependencies first?  
Did I choose sum, min, or max for the correct reason?  
Are impossible states represented safely?  
Do I need modulo, and only for counting?  
Do I need long long in C++?
```

18 Exercises

For A write base cases before transitions, for B state what `dp[i]` optimizes, and for C verify whether the last move is sufficient state. Test the smallest index, unreachable states, and equal-cost transitions.